МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №4
по курсу «Операционные системы»

Выполнил: К. Смирнов
Группа: М8О-201БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: освоение принципов работы с файловой системой и обмена данными между процессами через отображаемые файлы (File Mapping).

Задание: составить и отладить программу на языке Си, которая создаёт дочерние процессы и организует взаимодействие через отображаемые файлы и/или системные сигналы. Предусмотреть обработку системных ошибок. Вариант выбирается из лабораторной работы №2.

Вариант 20, группа 5. Родитель создаёт два дочерних процесса. Первые две строки пользовательского ввода задают имена выходных файлов. Строки длиной больше 10 символов направляются второму процессу, остальные - первому. Дочерние процессы инвертируют строки и записывают результаты в свои стандартные потоки вывода, перенаправленные в файлы. Родитель и ребёнок представлены разными программами.

Метод решения

Собираются две программы: parent и child. Родитель запускает два экземпляра child через fork и execl. Перед запуском stdout каждого ребёнка перенаправляется в соответствующий выходной файл через dup2. Диагностика детей выводится в stderr, поэтому не попадает в результаты обработки.

Вместо двух каналов из ЛР2 создаются два временных обычных файла через mkstemp. Имена сразу удаляются через unlink; открытые дескрипторы сохраняют доступ к файлам и наследуются через fork/exec. Каждый ребёнок получает номер своего дескриптора, PID родителя и свой порядковый номер в аргументах командной строки.

Для каждой строки родитель определяет адресата, вызывает ftruncate и mmap с MAP_SHARED, записывает заголовок и текст, снимает отображение и отправляет ребёнку SIGUSR1. Ребёнок отображает этот же файл, разворачивает текст, пишет результат, снимает отображение и подтверждает обработку через SIGUSR2. До подтверждения родитель следующую строку не передаёт.

Размер файла равен sizeof(Message) плюс фактическое количество байт текста. Поэтому у строк нет фиксированного ограничения длины. getline увеличивает входной буфер при необходимости. Конечный перевод строки хранится отдельным признаком и не участвует ни в подсчёте длины, ни в развороте.

Входные строки предполагаются английскими (ASCII), поэтому каждый символ занимает один байт. Длина определяется из результата getline без конечного LF. Для разворота меняются местами симметричные байты до середины строки.

Пустая строка отличается от завершения ввода: при EOF родитель отправляет отдельный пакет stop=1 обоим детям. Затем waitpid собирает их статусы. Дети существуют одновременно, но обработка строк в этой простой схеме последовательна. Время обработки строки линейно по её размеру, O(n).

Описание программы

include/ipc.h - структура Message и объявления функций обмена.

src/ipc.c - создание файлов, изменение размера, mmap/munmap, ожидание сигналов и полная запись write_all.

include/text.h, src/text.c - побайтовый разворот строк ASCII.

src/parent.c - ввод имён файлов, создание детей, фильтрация строк, отправка пакетов, ожидание и освобождение ресурсов.

src/child.c - получение пакетов, обработка, вывод результата и подтверждение.

CMakeLists.txt - сборка common, parent и child стандартным компилятором C11.

run_demo.sh, run_strace.sh - воспроизводимый запуск и сохранение трассы; tests/run_tests.py - проверки.

Системные вызовы и функции POSIX

Вызов / функция	Назначение
fork, execl	Создание процесса и замена его программы.
open, dup2, close	Открытие файлов и перенаправление stdout.
mkstemp, unlink	Уникальные временные файлы без оставшихся имён.
ftruncate, fstat	Размер IPC-файла и проверка выходных файлов.
mmap, munmap	Отображение файла и снятие отображения.
sigprocmask, sigwaitinfo	Блокировка и синхронный приём сигналов.
kill, waitpid	Уведомление процессов и сбор статусов.
write	Запись результата, включая частичные записи.

Синхронизация и обработка ошибок

SIGUSR1, SIGUSR2 и SIGCHLD блокируются до fork. Маска сохраняется через exec: ранний сигнал остаётся ожидающим. sigwaitinfo проверяет PID отправителя. Одновременно не бывает нескольких неподтверждённых передач, поэтому обычные сигналы не теряются из-за объединения.

Изменение размера допускается только после munmap ребёнка и SIGUSR2. Это исключает перезапись данных и доступ через отображение к уменьшенному файлу. msync не требуется для временного обмена: он управляет сбросом на носитель и не заменяет синхронизацию процессов.

Возвраты системных вызовов проверяются. При EINTR ожидание или запись повторяется. Если ребёнок завершился до подтверждения, SIGCHLD прерывает протокол с ошибкой. В cleanup родитель завершает оставшихся детей, выполняет waitpid, закрывает дескрипторы и освобождает буферы. Один и тот же выходной inode под разными именами отвергается до очистки.

Результаты

Среда: Ubuntu 24.04.1 LTS в WSL2, Linux 6.6.87.2-microsoft-standard-WSL2, GCC 13.3.0, strace 6.8. Сборка с -Wall -Wextra -Wpedantic прошла без предупреждений. Команды:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./run_demo.sh
./run_strace.sh
python3 tests/run_tests.py
```

Вход input.txt: первые две строки задают short.txt и long.txt. Ниже приведены данные и фактический результат. Пустая строка тоже обрабатывается.

Вход	Символов	Получатель	Результат
hello	5	child1	olleh
1234567890	10	child1	0987654321
12345678901	11	child2	10987654321
operating systems	17	child2	smetsys gnitarepo
(пустая строка)	0	child1	(пустая строка)
Hello OS	8	child1	SO olleH
abcdefghij	10	child1	jihgfedcba
abcdefghijk	11	child2	kjihgfedcba

Вывод программы при запуске с input.txt. Разные PID показывают создание отдельных процессов; порядок диагностических сообщений может меняться от запуска к запуску.

```
parent: PID=37042
File for child1 (length <= 10):
File for child2 (length > 10):
parent: started child1, PID=37043
parent: started child2, PID=37044
child1: PID=37043, parent=37042
parent: 5 chars -> child1
child2: PID=37044, parent=37042
child1: processed 5 bytes
parent: 10 chars -> child1
child1: processed 10 bytes
parent: 11 chars -> child2
child2: processed 11 bytes
parent: 17 chars -> child2
child2: processed 17 bytes
parent: 0 chars -> child1
child1: processed 0 bytes
parent: 8 chars -> child1
child1: processed 8 bytes
parent: 10 chars -> child1
child1: processed 10 bytes
parent: 11 chars -> child2
child2: processed 11 bytes
parent: child PID=37043, exit=0
parent: child PID=37044, exit=0
```

Скрипт run_demo.sh сравнил оба фактических файла с эталонами через cmp. Различий нет.

Проверка граничных случаев

Проверки запускались автоматически. Эталон вычисляется независимо в Python; проверяется полное содержимое обоих файлов. Тайм-аут обнаруживает зависания. После завершения дополнительно проверяется отсутствие созданных детей в /proc и новых имён IPC-файлов в /tmp.

```
PASS: length 0, 1, 9, 10, 11 and spaces
PASS: ASCII letters, punctuation and tabs
PASS: last line without newline
PASS: EOF without data
PASS: line of 1,000,000 characters
PASS: 500 random lines, five repeated runs
PASS: missing file names
PASS: invalid output path
PASS: same output file
PASS: hardlink alias rejected before truncation
PASS: child write error /dev/full, no deadlock
PASS: forced child termination, cleanup of both children
ALL 12 TESTS PASSED; no child processes or IPC paths left
```

Для строки на 1 000 000 символов проверена работа без фиксированного буфера. Пять запусков по 500 случайных строк проверяют маршрутизацию, порядок и отсутствие потерь. Отдельно проверены пробелы, табуляция, знаки препинания, отсутствие конечного LF, одинаковые выходы и жёсткая ссылка на один файл.

При записи в /dev/full дочерний write возвращает ENOSPC. Ребёнок завершает работу без подтверждения; родитель замечает SIGCHLD и не зависает. Отдельно первый ребёнок был принудительно остановлен SIGKILL. Настоящий лог этой проверки:

```
parent: PID=37406
File for child1 (length <= 10):
File for child2 (length > 10):
parent: started child1, PID=37407
parent: started child2, PID=37408
parent: 5 chars -> child1
parent: child PID=37407 exited before acknowledgement
parent: child PID=37407, signal=9
parent: child PID=37408, signal=15
```

Выводы

После этой лабораторной работы я стал внимательнее продумывать порядок обмена данными между процессами. При использовании отображаемого файла пришлось отдельно определить, когда ребёнок может читать строку и когда родитель может записывать следующую. Для этого используются сигналы готовности и подтверждения. Проверка с аварийным завершением ребёнка показала, что нужно учитывать и такой случай: родитель должен прекратить ожидание, завершить оставшийся процесс и освободить ресурсы.

Исходная программа: отображаемый файл

Приведены ключевые фрагменты. Полные исходники находятся в src/ и include/. Message не содержит указателей; text - гибкий массив после заголовка.

```
 7 typedef struct {
 8     size_t length;
 9     int stop;
10     int newline;
11     char text[];
12 } Message;
```

src/ipc.c: создание временного файла. После unlink доступ сохраняется через открытые дескрипторы.

```
22 int create_mapping(void) {
23     char name[] = "/tmp/lab4-XXXXXX";
24     int fd = mkstemp(name);
25     if (fd == -1) {
26         perror("mkstemp");
27         return -1;
28     }
29     if (unlink(name) == -1) {
30         perror("unlink");
31         close_fd(fd);
32         return -1;
33     }
34     return fd;
35 }
```

src/ipc.c: запись пакета в общий файл. Перед этим предыдущая обработка уже подтверждена ребёнком.

```
37 int put_message(int fd, const char *text, size_t len, int newline, int stop) {
38     if (len > SIZE_MAX - sizeof(Message)) {
39         errno = EOVERFLOW;
40         perror("message size");
41         return -1;
42     }
43     size_t size = sizeof(Message) + len;
44     off_t file_size = (off_t)size;
45     if (file_size < 0 || (size_t)file_size != size) {
46         errno = EOVERFLOW;
47         perror("file size");
48         return -1;
49     }
50     if (ftruncate(fd, file_size) == -1) {
51         perror("ftruncate");
52         return -1;
53     }
54     Message *msg = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
55     if (msg == MAP_FAILED) {
56         perror("mmap parent");
57         return -1;
58     }
59     msg->length = len;
60     msg->stop = stop;
61     msg->newline = newline;
62     if (len != 0) {
63         memcpy(msg->text, text, len);
64     }
65     if (munmap(msg, size) == -1) {
66         perror("munmap parent");
67         return -1;
68     }
69     return 0;
70 }
```

Исходная программа: запуск и ожидание

src/parent.c: создание отдельной программы ребёнка и перенаправление stdout. Номер fd передаётся аргументом, сам дескриптор наследуется.

```
 41 static pid_t start_child(int number, int maps[2], int outputs[2]) {
 42     pid_t pid = fork();
 43     if (pid != 0) {
 44         if (pid == -1) {
 45             perror("fork");
 46         }
 47         return pid;
 48     }
 49     if (dup2(outputs[number], STDOUT_FILENO) == -1) {
 50         perror("dup2");
 51         _exit(EXIT_FAILURE);
 52     }
 53     if (close_fd(outputs[0]) == -1 || close_fd(outputs[1]) == -1 ||
 54         close_fd(maps[1 - number]) == -1) {
 55         _exit(EXIT_FAILURE);
 56     }
 57     char fd_text[32], parent_text[32], num_text[32];
 58     snprintf(fd_text, sizeof(fd_text), "%d", maps[number]);
 59     snprintf(parent_text, sizeof(parent_text), "%d", getppid());
 60     snprintf(num_text, sizeof(num_text), "%d", number + 1);
 61     execl(CHILD_PATH, "child", fd_text, parent_text, num_text, (char *)NULL);
 62     perror("execl child");
 63     _exit(EXIT_FAILURE);
 64 }
```

src/ipc.c: синхронное ожидание уведомления. Родитель дополнительно ожидает SIGCHLD, чтобы обнаружить преждевременный выход ребёнка.

```
 98 int wait_signal(int signal_number, pid_t sender, int watch_children) {
 99     sigset_t set;
100     if (sigemptyset(&set) == -1 || sigaddset(&set, signal_number) == -1 ||
101         (watch_children && sigaddset(&set, SIGCHLD) == -1)) {
102         perror("signal set");
103         return -1;
104     }
105     for (;;) {
106         siginfo_t info;
107         int sig = sigwaitinfo(&set, &info);
108         if (sig == -1) {
109             if (errno == EINTR) {
110                 continue;
111             }
112             perror("sigwaitinfo");
113             return -1;
114         }
115         if (sig == SIGCHLD) {
116             if (info.si_code == CLD_EXITED || info.si_code == CLD_KILLED ||
117                 info.si_code == CLD_DUMPED) {
118                 fprintf(stderr, "parent: child PID=%d exited before acknowledgement\n",
119                         info.si_pid);
120                 return -1;
121             }
122             continue;
123         }
124         if (info.si_pid == sender) {
125             return 0;
126         }
127     }
128 }
```

Исходная программа: обработка строки

src/child.c: после SIGUSR1 ребёнок отображает пакет, обрабатывает и записывает строку. Подтверждение отправляется только после munmap.

```
36     for (;;) {
37         if (wait_signal(SIGUSR1, (pid_t)parent, 0) == -1) {
38             result = EXIT_FAILURE;
39             break;
40         }
41         size_t size;
42         Message *msg = get_message(fd, &size);
43         if (msg == NULL) {
44             result = EXIT_FAILURE;
45             break;
46         }
47         int stop = msg->stop;
48         if (!stop) {
49             reverse_text(msg->text, msg->length);
50             if (write_all(STDOUT_FILENO, msg->text, msg->length) == -1 ||
51                         (msg->newline && write_all(STDOUT_FILENO, "\n", 1) == -1)) {
52                 result = EXIT_FAILURE;
53             }
54             fprintf(stderr, "child%d: processed %zu bytes\n", number, msg->length);
55         }
56         if (munmap(msg, size) == -1) {
57             perror("munmap child");
58             result = EXIT_FAILURE;
59         }
60         if (stop || result != EXIT_SUCCESS) {
61             break;
62         }
63         if (kill((pid_t)parent, SIGUSR2) == -1) {
64             perror("kill parent");
65             result = EXIT_FAILURE;
66             break;
67         }
68     }
69     if (close_fd(fd) == -1 || close_fd(STDOUT_FILENO) == -1) {
70         result = EXIT_FAILURE;
71     }
72     return result;
73 }
```

src/text.c: разворот строки ASCII одним циклом. Обмениваются байты на позициях i и len-i-1; конечный LF передаётся отдельно.

```
3 void reverse_text(char *text, size_t len) {
4     for (size_t i = 0; i < len / 2; i++) {
5         char tmp = text[i];
6         text[i] = text[len - i - 1];
7         text[len - i - 1] = tmp;
8     }
9 }
```

Strace

Команда запуска и полная трасса сохранены в проекте. -f включает детей, -yy показывает файлы по дескрипторам, -tt добавляет время. Ниже неизменённый фрагмент strace.log для строки 1234567890; переносы длинных строк сделаны только при вёрстке.

```
strace -f -tt -yy -s 160 -o strace.log \
  ./build/parent < input.txt
```

```
37094 19:43:30.623908 ftruncate(5</tmp/lab4-RyBJDL>(deleted), 26) = 0
37094 19:43:30.624008 mmap(NULL, 26, PROT_READ|PROT_WRITE, MAP_SHARED, 5</tmp/lab4-RyBJDL>(deleted), 0)
= 0x781b89603000
37094 19:43:30.624097 munmap(0x781b89603000, 26) = 0
37094 19:43:30.624206 kill(37095, SIGUSR1) = 0
37095 19:43:30.624261 <... rt_sigtimedwait resumed>{si_signo=SIGUSR1, si_code=SI_USER, si_pid=37094,
si_uid=1000}, NULL, 8) = 10 (SIGUSR1)
37094 19:43:30.624279 write(1</dev/null<char 1:3>>, "parent: 10 chars -> child1\n", 27 <unfinished ...>
37095 19:43:30.624307 fstat(5</tmp/lab4-RyBJDL>(deleted),  <unfinished ...>
37094 19:43:30.624318 <... write resumed>) = 27
37094 19:43:30.624348 rt_sigtimedwait([USR2 CHLD],  <unfinished ...>
37095 19:43:30.624364 <... fstat resumed>{st_mode=S_IFREG|0600, st_size=26, ...}) = 0
37095 19:43:30.624402 mmap(NULL, 26, PROT_READ|PROT_WRITE, MAP_SHARED, 5</tmp/lab4-RyBJDL>(deleted), 0)
= 0x7ce2975a0000
37095 19:43:30.624494 write(1</home/konstsmir/os_labs/lab4/short.txt>, "0987654321", 10) = 10
37095 19:43:30.624579 write(1</home/konstsmir/os_labs/lab4/short.txt>, "\n", 1) = 1
37095 19:43:30.624668 write(2</dev/null<char 1:3>>, "child1: processed 10 bytes\n", 27) = 27
37095 19:43:30.624763 munmap(0x7ce2975a0000, 26) = 0
37095 19:43:30.624850 kill(37094, SIGUSR2 <unfinished ...>
37094 19:43:30.624908 <... rt_sigtimedwait resumed>{si_signo=SIGUSR2, si_code=SI_USER, si_pid=37095,
si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
37095 19:43:30.624940 <... kill resumed>) = 0
```

PID 37094 - родитель, 37095 - первый ребёнок. ftruncate устанавливает 26 байт: 16 заголовка и 10 текста. Родитель отображает файл с MAP_SHARED, заполняет его, снимает отображение и отправляет SIGUSR1. Ребёнок отображает тот же файл по своему адресу, пишет 0987654321 в short.txt, снимает отображение и возвращает SIGUSR2. Затем родитель продолжает ввод.

sigwaitinfo в Linux виден как rt_sigtimedwait с NULL-тайм-аутом; waitpid в конце полного лога - как wait4. fork представлен clone, execl - execve. Записи unfinished/resumed - части одного вызова при переключении между процессами. memcpy и разворот в трассе отсутствуют: это операции над памятью без обращения к ядру.

Первичные справочники: mmap(2), ftruncate(2), sigwaitinfo(2), sigprocmask(2), execve(2) - Linux man-pages.